INVENTORY MANAGEMENT SYSTEM: CONTROL, TRENDS AND LOW STOCK ALERTS

TABLE OF CONTENTS

1. Cover Page
- 1.1 Project Title
- 1.2 Author(s)
- 1.3 University / Course / Date

2. Executive Summary

3. Introduction
- 3.1 Context and Problem Statement
- 3.2 Project Objectives

4. Context, Problem, and Solution
- 4.1 Description of the problem in microenterprises
- 4.2 Consequences of manual management
- 4.3 Proposed solution

5. System Architecture
- 5.1 Overview of modules
- 5.2 Conceptual diagram

6. Module Details
- 6.1 **base.py — Database connection and setup**
  - 6.1.1 Function conectar_db()
  - 6.1.2 Function crear_tabla()
  - 6.1.3 Function funprecio()
  - 6.1.4 Observations and improvements
- 6.2 **inventario.py — Inventory logic and utilities**
  - 6.2.1 convertir_float_precio()
  - 6.2.2 generar_id_unico()
  - 6.2.3 agregar_producto()
  - 6.2.4 consultar_por_id() and consultar_por_nombre()
  - 6.2.5 listar_productos() and update functions
  - 6.2.6 General observations
- 6.3 **ventas.py — Sales logic**
  - 6.3.1 vender_producto_por_nombre()
  - 6.3.2 vender_producto_por_id()
- 6.4 **main.py — Graphical User Interface (GUI)**
  - 6.4.1 General structure (CTkTabview)
  - 6.4.2 Tab descriptions
    - Inventory

• Update
• Sales
• Query
• Trends
• Delete
6.4.3 Key functions
6.4.4 Detected errors and corrections

7. SQL Functionality in the System

8. Testing and Validation
8.1 Unit tests
8.2 Integration tests
8.3 User Acceptance Testing (UAT)

9. Security and Best Practices
9.1 Error handling and logging
9.2 Transactions and rollbacks
9.3 Input validation

10. Recommendations and Improvement Plan
10.1 Immediate corrections
10.2 Short/medium-term improvements
10.3 Long-term evolution

11. System Execution Guide

12. Flowchart

13. Conclusion

14. Next Steps

15. Annexes and Appendices
A. Code snippets
B. SQL schema
C. Function glossary

16. Authorship and Contact Information

Detailed explanation of the project
This project is a simple **inventory system** with a graphical interface (CustomTkinter) and a SQLite database. It is organized into 4 main modules:

- base.py → database connection and preparation functions.

base

- inventario.py → functions for adding/querying/updating/listing products (business logic).

inventory

- ventas.py → functions to make sales (update stock and calculate totals).

sales

- main.py → graphical interface and "glue" between the GUI and the previous functions.

main

Below I detail **each file**, what each function/method does, why certain things are used and possible improvements/errors to be fixed.

---

base.py — Connecting and preparing the database

base

Main functions:

1. conectar_db()
    - Returns sqlite3.connect('inventario.db'). It is the input for all operations with the database.
    - Comment: The inventario.db database is created in the execution directory if it does not exist.
2. crear_tabla()
    - Open connection, create the products table if it does not exist with columns:
        - id TEXT PRIMARY KEY
        - UNIQUE TEXT number
        - INTEGER quantity
        - precio REAL DEFAULT 0
    - Use IF NOT EXISTS so you don't crash if it already exists. Make commit() and close().
    - Note: there is no explicit handling of exceptions; If the creation fails for some reason, the exception will reach the caller.
3. funprecio()
    - See the table structure with PRAGMA table_info(products) for columns.
    - If the "price" column is missing, run ALTER TABLE products ADD COLUMN price REAL DEFAULT 0.
    - Purpose: Simple migration that ensures the price column exists (useful if the table was created by an earlier version).
    - Notes on SQLite: ALTER TABLE ADD COLUMN is supported; other operations (drop column) are not easily supported. funprecio() should be called **after** crear_tabla() (and in fact it is done in main.py).

Good practices/possible improvements in base.py:

- Use **context managers** (with sqlite3.connect(...) as conn:) to ensure automatic commit()/close() and exception handling.
- Have version control of migrations instead of ad-hoc PRAGMA (e.g. a migrations table or use Alembic/simple migrations).
- Add logs or more descriptive errors if the DB cannot be opened or the table does not exist.

How function sql in our code

First, we import sqlite 3

The with the code line

```
def conectar_db():
    return sqlite3.connect('inventario.db')
```

So, if not exist create the data base and if it exists return

Now with .execute() we use sql code in phyton
Create table in sql:

```python
def crear_tabla():
    conexion = conectar_db()
    cursor = conexion.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS productos (
            id TEXT PRIMARY KEY,
            nombre TEXT UNIQUE,
            cantidad INTEGER,
            precio REAL DEFAULT 0
        )
    ''')
    conexion.commit()
    conexion.close()
```

Here we create the table and the columns with the id,name,amount

And in the functions, we create the Conexion whit the data base and a good practice is close the Conexion
Example:

```python
def funprecio():
    conexion = conectar_db()
    cursor = conexion.cursor()
    cursor.execute("PRAGMA table_info(productos)")
    cols = [row[1] for row in cursor.fetchall()]
    if "precio" not in cols:
        cursor.execute("ALTER TABLE productos ADD COLUMN precio REAL DEFAULT 0")
        conexion.commit()
    conexion.close()
```

In every function we use sql code for add information and updated in the data base

inventario.py — Inventory and Profit Logic (Detailed)
inventory
convertir_float_precio(text)

- **What it does:** normalizes different price formats (e.g.: '12.50', '12.50', '$12,500', '12,500.99', '12,500.99') and returns a float with 2 decimal places (round(..., 2)), or None if it can't convert.

- **How it works internally (step by step):**
    1. If text is None, it returns None.
    2. Eliminate $ and spaces.
    3. If it contains **',' and '.'**: 12.345.67 format is assumed → first deletes the dots (thousands), then converts the comma to a period (decimal) → '12345.67'.
    4. If it contains **','** but not '.', replace the comma with a period (decimal).
    5. If it contains only one. and the part after the period has 3 digits (e.g. '12.500'), interprets the. as a separator of thousands and erases it; so '12,500' → '12500'.
    6. Try float(s) and round(value,2). If it fails, return None.

- **Examples:**
    - '1,234.56' → 1234.56
    - '12.5' → 12.5 → 12.50 (rounded)
    - '$ 12 500.99' → 12500.99
- **Limitations/ambiguous cases:** '12.50' (dollar with decimal or thousands?) is assumed decimal; '1.234' will be interpreted as 1234 only if the decimal part has 3 digits (heuristics). These heuristics work for many local formats, but they don't cover all cases; document it for users.

generar_id_unico(number)

- **What it does: Generate** a short id (8 hex) using name MD5 + timestamp.
- **Notes:**
    - It's a quick way to create readable, low-collision probability IDs.
    - Best alternative for strong uniqueness: uuid4() or combine with a counter. MD5 here is fine for a local app.

agregar_producto(name, quantity, price=None)

- **Flow:**
    1. Connect to DB.
    2. Find if there is already SELECT id, quantity FROM products WHERE name = ?.
    3. If it exists: Update the quantity (add the new quantity to the existing one) and return the existing id.
    4. If it does not exist: generate id_producto with generar_id_unico() and insert INSERT INTO products (id, name, quantity, price).
    5. commit() y close(). Retorna id_producto o None en IntegrityError.

- **Important observations:**
    - If price is None, inserting sends None to the INSERT. In SQLite that will leave the price column null (although the table has DEFAULT 0, the DEFAULT is not applied when NULL is explicitly specified). If you want to force 0.0 when you don't pass price, you should normalize first: price = price if price is not None else 0.0.
    - Error handling: sqlite3 is captured. IntegrityError to detect duplicates, but other errors (e.g., disk problems) are not caught.
    - Inconsistent returns: sometimes returns id (string), other times prints and returns None. This can make it difficult for the GUI to use.

consultar_por_id(identifier)

- They return a **dictionary** with the fields if they find the product, or a **string** with an error message if they can't find it ("ID not found" / "Product not found").
- **Note:** mixing return types (dict vs str) forces the caller to check the type; better to throw None or an exception or always return None/dict for consistency.

listar_productos()

- Retrieves all products (SELECT id, name, quantity, price FROM products ORDER BY name) and **prints them by console** (does not return the list).

- In main.py a function of its own is made refrescar_lista() that queries the DB and displays in the GUI, so this function is more useful for console or debug use.

actualizar_por_id(here, nuevo_precio)

- They check if the record exists (using COUNT(*)) and then update the price column. They return a message (string) in case of success.
- **Possible pitfall:** nuevo_precio in the main.py code is sometimes passed as **a string** (Entry text) and not as a float. SQLite will accept it (storing it as is), but it's better to validate/convert before the call (use convertir_float_precio and pass a float).

General comments on inventario.py

- In inventario.py is done from base import conectar_db and also from base import sqlite3. Importing sqlite3 from base is unusual; It is preferable to import SQLite3 directly into this module.
- There is a mixture of **print per console** (print) and **returns**: it is convenient to define a style (return data and let the GUI display messages) for separation of responsibilities.
- I recommend using with conectar_db() as connection: and handling more specific exceptions.

---

ventas.py — Sales Logic (Explained)
sales
Functions:
vender_producto_por_nombre(name, quantity)

- **Flow:**
    1. Connect and SELECT quantity, price FROM products WHERE name = ?.
    2. If it does not exist→ print "Product not found" and exit.
    3. If you > stock quantity → prints out of stock message and exits.
    4. Subtract the stock (nuevo_stock = stock - quantity), calculate total = quantity * price.
    5. Update the table: UPDATE products SET quantity = ? WHERE name = ?, commit(), and close().
    6. Prints summary of the sale (quantity, unit price, total, remaining stock).

vender_producto_por_id(identifier, quantity)

- Same logic as by name, but using id in the query and update.

GUI Structure

main.py — Graphical Interface and Final Flow
main
Beginning

- Import customtkinter (you need to install the library) and module functions.
- Call crear_tabla() and funprecio() at boot to prepare the DB.
- Use CTkTabview to create tabs: **Inventory**, **Update**, **Sales**, **Inquiry**, **Trends**, **Delete**.
- Each tab has Entry, Button, Label, Textbox, and buttons that call functions defined in the same file (accion_agregar, accion_consultar_por_id, etc.).

Important features in main.py

refrescar_lista()

- Query the DB (SELECT id, name, quantity, price FROM products ORDER BY name) and type the result in txt_lista (a CTkTextbox) to display inventory in the GUI. Handle the empty case.

accion_agregar()

- Read entry_nom, entry_cantidad, entry_precio and validate:
  - Convert quantity to int and demand > 0.
  - Convert precio_raw with convertir_float_precio.
- Call agregar_producto(name, quantity, price) and if id is generated, display message in lbl_salida, clean entries and refresh list.

- **Bugs/inconsistencies detected:**
  - After adding try entry_nombre.delete(0, "end") instead of entry_nom.delete(...). entry_nombre is the **query field by name**; this looks like a copy/paste error. The field that should be cleaned up is entry_nom.
  - In case of error, messagebox.showerror is used. Correct, but the input and label cleanup flow has variable name inconsistencies in several places (see below).

accion_consultar_por_id()

- They call the consultar_por_id is a function and display the result in tags.

- **Note:** the result is assigned to lbl_consultar_por_id.configure(...)

accion_consulta()

- Generic field where the user can enter ID; The function first tries to search by ID. Insert the result into txt_resultado.
- **Note:** The name is not transformed to lower() here; if the names in the DB are saved in lowercase (as is done in accion_agregar), the name search may fail if the user does not type lowercase.

accion_buscar() and accion_modificar()

- accion_buscar() detects if the text is isalpha() (letters only) takes it as a name; If isalnum() takes it as id.

- **Problem:** isalpha() and isalnum() are fragile conditions: they don't allow spaces, hyphens, accents, or special characters. A product name like "fries" (with space) would cause validation to fail. Better: always try to consultar_por_id(value) and, if there are no results, consultar_por_nombre(value) (try both).

- accion_modificar() validates the price using convertir_float_precio but **then calls** actualizar_por_nombre(name, nuevo_precio) by passing nuevo_precio **in text** (string), not the validated float. The converted price (float) must be passed, not the str. This can cause the DB to receive text or NULL.

Sales (accion_vender_por_nombre, accion_vender_por_id)

- Validate quantity and call vender_producto_por_nombre or vender_producto_por_id.

- **Note:** The name call does not .lower() the name before calling the sales function; if the names in the DB are normalized to lowercase, the search may fail.

Delete products

- accion_eliminar_tab() and accion_eliminar_id_tab() execute DELETE FROM products WHERE name = ? / WHERE id = ? and they use cursor.rowcount to know how many rows were deleted; refresh the list and display messages. This is fine (use parameterized queries, thus avoiding SQL injection).

Trending tab

- It currently has inputs and buttons, **but they are not connected** to functions that calculate trends. They are placeholders.

UI errors / inconsistencies detected (summary)

- Misused GUI variables (e.g., clean entry_nombre instead of entry_nom, use lbl_consultar_por_id in the query function by name). Find and correct those copy/paste.
- Pass nuevo_precio as str to functions that wait for number: always use price = convertir_float_precio(text) and pass price (float) to the update.
- Validations with isalpha()/isalnum() are not robust for real names (spaces, symbols). Best: Try both queries (id → name).
- Mix print() in modules and messages in GUI: centralize the presentation of messages in the GUI and let the modules return results/errors.

Technical recommendations and suggested improvements

1. **Consistency of Types and Returns**
    - Normalize that inventario.py functions return dict/None or throw exceptions in case of error (avoid mixing print and strings as messages).

2. **Usar context managers para la BD**
    - with conectar_db() as conn: or directly with sqlite3.connect(...) as conn: to secure commits/rollbacks and closure.

3. **Money management**
    - Avoid float for money in serious apps; use Decimal (from decimal) or store pennies such as INTEGER.

4. **Record Sales**
    - Create sales table (id, producto_id, quantity, precio_unitario, total, date) to be able to generate reports and trends.

5. **GUI Validations**
    - Fix poorly referenced Entry/Label.
    - Normalize names: when saving name.lower() — then when searching to use .lower(); or better, save and search with COLLATE NOCASE in SQLite: WHERE name = ? COLLATE NOCASE.

6. **Migrations**
    - Replace funprecio() with a simple migration/versioning system if you plan to evolve the DB.

7. **Improve IDs**
    - uuid.uuid4().hex[:8] or keep MD5 if it's okay for local scope. UUID4 is more standard.

8. **Interface: user feedback**
    - Have business functions return structures with success, message, data so that the GUI decides how to display messages.

9. **Internationalization and formats**
    - convertir_float_precio does a good job; It documents the accepted formats and adds unit tests for that function.

How to Run:

1. Instalar dependencias:

```bash
pip install customtkinter
```

(si no tienes `customtkinter`, usa `pip` —en algunos entornos la instalación puede variar)

2. Ejecutar:

```bash
python main.py
```

- Se creará `inventario.db` en el directorio actual.
- Asegúrate de tener los 4 archivos en el mismo directorio.

Imports used:

custotkinter as ctk:

- It is a library that extends tkinter with modern widgets (flat buttons, tabs, dark colors, etc.).
- It is imported as ctk for short.
- Example: ctk. CTk() instead of tk. Tk() creates the main window with a modern style.

from tkinter import messagebox:

- MessageBox is a standard Tkinter module that displays **pop-ups** (modal dialogs) for messages.
- It is used to display errors, warnings, or confirmations to the user.
- Example:

```python
messagebox.showerror("Error", "Cantidad inválida")
```

```
import customtkinter as ctk
from tkinter import messagebox
from base import crear_tabla, funprecio
from inventario import (
    agregar_producto, consultar_por_id, consultar_por_nombre,
    listar_productos, actualizar_por_id, actualizar_por_nombre, convertir_float_p
)
from ventas import vender_producto_por_id, vender_producto_por_nombre
import sqlite3
```

This opens a window with a ❌ and the error text.

- **from base import crear_tabla, funprecio**:
    - Import functions from the database.
    - crear_tabla() → ensures that the Products table exists.
    - funprecio() → adds price column if missing (compatibility).
    - They are run at the start of the GUI to prepare the DB.
- **from inventory import (...)**:

    - Import business logic functions: add, query, list, update, and convert prices.
    - They are the ones that connect the interface to the storage in SQLite.
- **from ventas import vender_producto_por_id, vender_producto_por_nombre**:
    - Functions for sales from the GUI.
- **import sqlite3**:
    - It is imported to run some direct queries from the GUI (e.g. refresh list, delete).
    - Although in principle the GUI should always use inventario.py and ventas.py, here it is done directly for simplicity.

```python
ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

ventana = ctk.CTk()
ventana.title("Sistema de Inventario")
ventana.geometry("900x600")
```

- **ctk.set_appearance_mode("dark")**: cambia el tema general de los widgets a oscuro.
- **ctk.set_default_color_theme("blue")**: Defines the main color of buttons and highlights.
- **window = ctk. CTk()**: Creates the **main application window** .
- **.title("...")**: Puts the title of the window.
- **.geometry("900x600")**—Defines the initial size in pixels (900 width x 600 height).

Tab (CTkTabview)

```python
tabview = ctk.CTkTabview(ventana)
tabview.pack(expand=1, fill="both")
```

- **CTkTabview**: Creates a tabbed container (similar to a notebook).
- pack(expand=1, fill="both"): Makes it fill the entire window, expand, and adjust automatically.

Then tabs are created with:Each variable (tab_inventario, tab_actualizar, etc.) is a **frame** that corresponds to each tab.

```python
tab_inventario = tabview.add("Inventario")
tab_actualizar = tabview.add("Actualizar")
tab_ventas = tabview.add("Ventas")
tab_consulta = tabview.add("Consulta")
tab_tendencias = tabview.add("Tendencias")
tab_eliminar = tabview.add("Eliminar")
```

Main graphical variables

(a) Inventory

- **Entry**:

```python
entry_nom = ctk.CTkEntry(tab_inventario)
entry_cantidad = ctk.CTkEntry(tab_inventario)
entry_precio = ctk.CTkEntry(tab_inventario)
```

Text fields where the user enters the name, quantity, and price of the product.

Button:

btn_agregar = ctk. CTkButton(tab_inventario, text="Add", command=accion_agregar)

A button that when clicked executes the accion_agregar() function.

Textbox for List:

```python
txt_lista = ctk.CTkTextbox(tab_inventario, width=500, height=200)
```

Displays all products in inventory (updated with refrescar_lista()).

Label Label:

python Copiar código

```python
lbl_salida = ctk.CTkLabel(tab_inventario, text="")
```

→ Muestra mensajes al usuario (ejemplo: "Producto agregado con éxito").

Update

- Tickets:
  - **entry_buscar** → to type ID or name.
  - **entry_nuevo_precio** → to write new price.
- Bellboy:
  - **btn_buscar** → looking for the product.
  - **btn_modificar** → update the price.
- Label:

  lbl_resultado_actualizar → shows search or update result

Sales

- Two sections: sale by **name** and sale by **ID.**

Example:

entry_nombre_vender = ctk. CTkEntry(tab_ventas)

entry_cantidad_vender = ctk. CTkEntry(tab_ventas)

btn_vender_nombre = ctk. CTkButton(tab_ventas, text="Vender", command=accion_vender_por_nombre)

Similar for ID: entry_id_vender, **entry_cantidad_vender_id**, btn_vender_id.

A label: **lbl_resultado_ventas** to show the result**.**

Consultation

- **entry_consulta → to type ID or name.**
- **btn_consulta → run search.**
- **txt_resultado → a Textbox where the result (product detail) is written.**

e) Trends

- Still no logic implemented (only sample inputs and buttons).

---

f) Eliminate

- **Two sections:** delete by name or by ID.
- **Variables: entry_eliminar_nombre, entry_eliminar_id**, associated buttons, and commit labels.

---

5. Interface Functions (Actions)

Example: **accion_agregar()**

```python
def accion_agregar():
    nombre = entry_nom.get()
    cantidad = entry_cantidad.get()
    precio_raw = entry_precio.get()

    # Validaciones
    if not nombre or not cantidad:
        messagebox.showerror("Error", "El nombre y la cantidad son obligatorios")
        return
```

entry_nom.get() → gets what the user typed in the text box.

messagebox.showerror(...) → if the user left empty fields, a **pop-up appears with a red icon** that interrupts execution.

Later:

```python
    try:
        cantidad = int(cantidad)
        if cantidad <= 0:
            raise ValueError
    except ValueError:
        messagebox.showerror("Error", "La cantidad debe ser un número positivo")
        return
```

- An attempt is made to convert the quantity to an integer.
- If it fails or is ≤0, it displays another messagebox.showerror.

Then validate price with convertir_float_precio(). If everything is fine:

```python
id_producto = agregar_producto(nombre.lower(), cantidad, precio)
if id_producto:
    lbl_salida.configure(text=f"Producto agregado con ID: {id_producto}")
    entry_nom.delete(0, "end")
    entry_cantidad.delete(0, "end")
    entry_precio.delete(0, "end")
    refrescar_lista()
```

lbl_salida.configure(...): Change the text of the label to give feedback.

delete(0, "end"): Cleans up the entries so the user can type something else.

refrescar_lista()— Updates the txt_lista to display the new product added.

Using a messagebox

In several functions (accion_agregar, accion_modificar, accion_vender...) messagebox.showerror is used.

Main messagebox options:

- messagebox.showinfo("Title", "Message") → blue/informational window.
- messagebox.showwarning("Title", "Message") → yellow/warning window.
- messagebox.showerror("Title", "Message") → red/error window.

It is used because it is more noticeable than just changing a Label, and **it forces the user to read it** before continuing.

General GUI flow

1. The window (window) is created.
2. Tabviews are created.
3. Each tab has its widgets (Entry, Label, Button, Textbox).
4. Each button has a command=... that connects to a function (accion_*).
5. Functions:

- They get data from the inputs with .get().
- Validate (show errors with messagebox if something goes wrong).
- They call logic functions (agregar_producto, consultar_por_id, etc.).
- Update tags or Textbox with .configure() or .insert().
- They clean up entries with .delete(0,"end") if necessary.
-

Errors detected in GUIs

- accion_agregar() cleans entry_nombre.delete(...), but the correct variable is entry_nom. This will cause a **NameError**.
- accion_consultar_por_nombre() displays the result in lbl_consultar_por_id instead of lbl_consultar_por_nombre. Confusion of labels.
- In accion_modificar(), you get price = convertir_float_precio(...) but you pass nuevo_precio (string) to the update function. That can put text into the DB.

Conclusion

The developed system implements a comprehensive solution for inventory management, sales and inquiries through a graphical interface built with **CustomTkinter** and a **SQLite** database. Its modular design, divided into files (base.py, inventario.py, ventas.py and main.py), allows a clear separation between business logic, database management and the presentation layer.

The interface offers intuitive tabs for adding, viewing, updating, selling, and deleting products, while the database ensures information persistence. In addition, auxiliary functions such as convertir_float_precio ensure flexibility in entering data in different formats.

Next steps

Although the system fulfills the basic functions of inventory management, there are still opportunities for improvement:

- Implement a **sales record** with dates and totals to generate reports.
- Optimize **data validation** in the interface, avoiding errors due to empty or poorly formatted entries.
- Add **trend graphs** in the corresponding tab.
- Include user authentication to control permissions in critical operations.
- Replace float in prices with Decimal or integers (cents) for accuracy.

Endnotes

This project was designed for educational and practical purposes. Its modular architecture makes it easy to extend functionality and integrate new features without altering the existing foundation.

The system is intended for **small local applications**, but can be scaled with improvements in database management, security, and reporting.

Authorship

Developed by: **Andrés Felipe Quintero Jaramillo**
Date: **2025**

Contact: aquinte48109@universidadean.edu.co

License

License: **MIT.** The use, modification and distribution of this software is permitted provided that the corresponding license notice is included. (use it if you want)